# Trabajo Práctico N° 5

Gestión de Contactos con Base de Datos en Java

Programación II

Licenciatura en Ciencias de la Computación

Gabriel Lopez Romero

# Índice

**1. Introducción** **2**

**2. Diseño de la Base de Datos y Conexión** **2**

**3. Operaciones CRUD y Consultas Vinculadas** **2**
3.1. Métodos Implementados . . . . . 2

**4. Manejo de Excepciones** **3**

**5. Implementación en Java y Pruebas** **4**
5.1. Clase GestorContactos . . . . . 4
5.2. Clase Main . . . . . 11

# 1. Introducción

El presente informe detalla la resolución del Trabajo Práctico número 5 de la cátedra Programación II. El objetivo principal de este trabajo es comprender el funcionamiento de una base de datos relacional utilizando **SQLite** e integrarla con los conceptos de la POO en Java mediante el driver **JDBC**, implementando operaciones CRUD, consultas vinculadas con JOIN y manejo de excepciones.

# 2. Diseño de la Base de Datos y Conexión

Se utilizó SQLite trabajando sobre el archivo local `agenda.sqlite` mediante la librería `sqlite-jdbc` gestionada con Maven.

- **Gestión de la conexión JDBC:** En la clase *GestorContactos*, se mantiene un atributo `Connection conn` que se inicializa al instanciar el objeto. Para ello se carga en memoria el driver mediante `Class.forName("org.sqlite.JDBC")` y se abre la conexión con `DriverManager.getConnection("jdbc:sqlite:agenda.sqlite")`. Además, se ejecuta la sentencia `PRAGMA foreign_keys = ON;` para que SQLite active la validación de claves foráneas, y se provee el método `cerrarConexion()` para liberar el recurso al finalizar la ejecución.
- **Estructura de tablas relacionadas:** El método `crearTablas()` elimina previamente las tablas si ya existen (`DROP TABLE IF EXISTS`) respetando el orden de las dependencias y luego crea las tres tablas solicitadas:
  - `contactos`: almacena `id` (clave primaria autoincremental), `nombre`, `apellido` y `telefono`.
  - `emails`: almacena `id`, `contacto_id` y `email`, definiendo `FOREIGN KEY (contacto_id) REFERENCES contactos(id)`.
  - `direcciones`: almacena `id`, `contacto_id` y `direccion`, definiendo `FOREIGN KEY (contacto_id) REFERENCES contactos(id)`.

# 3. Operaciones CRUD y Consultas Vinculadas

Todas las operaciones sobre la base de datos se encapsularon en la clase *GestorContactos*, combinando el uso de `Statement` para consultas estáticas y `PreparedStatement` para consultas parametrizadas con signos de pregunta (`?`), evitando posibles errores de sintaxis.

## 3.1. Métodos Implementados

- **CRUD de Contactos:**
  - `crearContacto(nombre, apellido, telefono)`: valida que los campos no estén vacíos, inserta el registro en la tabla `contactos` y devuelve el `id` autogenerado mediante `Statement.RETURN_GENERATED_KEYS`.

- `leerContactos()`: realiza una consulta vinculada entre las tres tablas utilizando `LEFT JOIN` sobre la columna `contacto_id` para mostrar cada contacto junto con sus emails y direcciones asociadas.
- `actualizarTelefono(id, nuevoTelefono)`: verifica la existencia del contacto y actualiza su número telefónico.
- `eliminarContacto(id)`: verifica que el contacto exista y elimina primero sus registros asociados en `emails` y `direcciones` antes de borrar el registro en `contactos`.

- **CRUD de Emails y Direcciones:** Se implementaron métodos equivalentes para gestionar los datos vinculados a un contacto: `crearEmail`, `leerEmails`, `actualizarEmail` y `eliminarEmail`, así como `crearDireccion`, `leerDirecciones`, `actualizarDireccion` y `eliminarDireccion`.

# 4. Manejo de Excepciones

Cada transacción está protegida mediante bloques `try-catch` que capturan `SQLException`, `ClassNotFoundException` y `Exception` general, contemplando los tres escenarios requeridos:

- **Conexión fallida a la base de datos:** Se controla en el constructor `GestorContactos(String url)` capturando `SQLException` cuando la ruta del archivo SQLite es inválida o inaccesible, y `ClassNotFoundException` si el driver JDBC no se encuentra cargado.
- **Inserción de datos inválidos:** El método auxiliar `validarTexto(valor, campo)` verifica que ningún texto ingresado sea `null` o esté vacío (`trim().isEmpty()`), lanzando una excepción descriptiva antes de ejecutar la sentencia SQL.
- **Acceso a un contacto o registro inexistente:** El método `verificarExisteContacto(contact` consulta si el `id` existe en la tabla `contactos` antes de asociar datos, modificar el teléfono o eliminar el contacto; en caso contrario (o cuando `executeUpdate()` afecta 0 filas), se lanza una `SQLException` informando que el registro no existe.

# 5. Implementación en Java y Pruebas

A continuación se adjunta el código implementado en Java para las clases *GestorContactos* y *Main*, seguido de la captura de pantalla con la salida obtenida al ejecutar todas las pruebas.

## 5.1. Clase GestorContactos

```java
package agenda;

import java.sql.*;

public class GestorContactos {

  private Connection conn = null;

  public GestorContactos() {
    this("jdbc:sqlite:agenda.sqlite");
  }

  public GestorContactos(String url) {
    try {
      Class.forName("org.sqlite.JDBC");
      conn = DriverManager.getConnection(url);
      if (conn != null) {
        Statement stmt = conn.createStatement();
        stmt.execute("PRAGMA foreign_keys = ON;");
        stmt.close();
        System.out.println("Conexión a base de datos ... Ok");
      } else {
        System.out.println("Conexión a base de datos fallida!");
      }
    } catch (SQLException a) {
      System.out.println("Error SQL de conexión: " + a.getMessage());
    } catch (ClassNotFoundException ex) {
      System.out.println("Error de driver: " + ex.getMessage());
    } catch (Exception ex) {
      System.out.println("Error: " + ex.getMessage());
    }
  }

  public void cerrarConexion() {
    try {
      if (conn != null && !conn.isClosed()) {
        conn.close();
      }
    } catch (SQLException a) {
      System.out.println("Error al cerrar conexión: " + a.getMessage());
    }
  }

  private void validarTexto(String valor, String campo) throws Exception {
    if (valor == null || valor.trim().isEmpty()) {
      throw new Exception("Dato inválido: el campo '" + campo + "' no puede estar
      ↪  vacío.");
    }
  }
```

```
49
50  private void verificarExisteContacto(int contactoId) throws SQLException {
51    PreparedStatement pstmt = conn.prepareStatement("SELECT id FROM contactos WHERE
      ↪  id = ?");
52    pstmt.setInt(1, contactoId);
53    ResultSet rs = pstmt.executeQuery();
54    boolean existe = rs.next();
55    rs.close();
56    pstmt.close();
57    if (!existe) {
58      throw new SQLException("No existe el contacto con id " + contactoId + ".");
59    }
60  }
61
62  public void crearTablas() {
63    try {
64      Statement stmt = conn.createStatement();
65
66      stmt.execute("DROP TABLE IF EXISTS direcciones;");
67      stmt.execute("DROP TABLE IF EXISTS emails;");
68      stmt.execute("DROP TABLE IF EXISTS contactos;");
69
70      stmt.execute(
71          "CREATE TABLE contactos ("
72              + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
73              + "nombre TEXT NOT NULL, "
74              + "apellido TEXT NOT NULL, "
75              + "telefono TEXT NOT NULL"
76              + ");");
77
78      stmt.execute(
79          "CREATE TABLE emails ("
80              + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
81              + "contacto_id INTEGER NOT NULL, "
82              + "email TEXT NOT NULL, "
83              + "FOREIGN KEY (contacto_id) REFERENCES contactos(id)"
84              + ");");
85
86      stmt.execute(
87          "CREATE TABLE direcciones ("
88              + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
89              + "contacto_id INTEGER NOT NULL, "
90              + "direccion TEXT NOT NULL, "
91              + "FOREIGN KEY (contacto_id) REFERENCES contactos(id)"
92              + ");");
93
94      stmt.close();
95    } catch (SQLException a) {
96      System.out.println("Error SQL: " + a.getMessage());
97    } catch (Exception ex) {
98      System.out.println("Error: " + ex.getMessage());
99    }
100 }
101
102 public int crearContacto(String nombre, String apellido, String telefono) {
103   int idGenerado = -1;
104   try {
105     validarTexto(nombre, "nombre");
```

```
106         validarTexto(apellido, "apellido");
107         validarTexto(telefono, "telefono");
108 
109         String sql = "INSERT INTO contactos (nombre, apellido, telefono) VALUES (?, ?,
            ↪  ?)";
110         PreparedStatement pstmt = conn.prepareStatement(sql,
            ↪  Statement.RETURN_GENERATED_KEYS);
111         pstmt.setString(1, nombre.trim());
112         pstmt.setString(2, apellido.trim());
113         pstmt.setString(3, telefono.trim());
114         pstmt.executeUpdate();
115 
116         ResultSet rs = pstmt.getGeneratedKeys();
117         if (rs.next()) {
118           idGenerado = rs.getInt(1);
119         }
120         rs.close();
121         pstmt.close();
122       } catch (SQLException a) {
123         System.out.println("Error SQL: " + a.getMessage());
124       } catch (Exception ex) {
125         System.out.println("Error: " + ex.getMessage());
126       }
127       return idGenerado;
128     }
129 
130     public void leerContactos() {
131       try {
132         String sql = "SELECT c.id, c.nombre, c.apellido, c.telefono, e.email,
            ↪  d.direccion "
133             + "FROM contactos c "
134             + "LEFT JOIN emails e ON c.id = e.contacto_id "
135             + "LEFT JOIN direcciones d ON c.id = d.contacto_id;";
136 
137         Statement stmt = conn.createStatement();
138         ResultSet rs = stmt.executeQuery(sql);
139 
140         boolean hayDatos = false;
141         while (rs.next()) {
142           hayDatos = true;
143           int id = rs.getInt("id");
144           String nombre = rs.getString("nombre");
145           String apellido = rs.getString("apellido");
146           String telefono = rs.getString("telefono");
147           String email = rs.getString("email");
148           String direccion = rs.getString("direccion");
149 
150           System.out.println(
151               "ID: "
152                   + id
153                   + " | "
154                   + nombre
155                   + " "
156                   + apellido
157                   + " | Tel: "
158                   + telefono
159                   + " | Email: "
160                   + email
```

```
                + " | Dirección: "
                + direccion);
      }
      if (!hayDatos) {
        System.out.println("No hay contactos cargados.");
      }

      rs.close();
      stmt.close();
    } catch (SQLException a) {
      System.out.println("Error SQL: " + a.getMessage());
    } catch (Exception ex) {
      System.out.println("Error: " + ex.getMessage());
    }
  }

  public void actualizarTelefono(int id, String nuevoTelefono) {
    try {
      validarTexto(nuevoTelefono, "telefono");
      verificarExisteContacto(id);

      String sql = "UPDATE contactos SET telefono = ? WHERE id = ?";
      PreparedStatement pstmt = conn.prepareStatement(sql);
      pstmt.setString(1, nuevoTelefono.trim());
      pstmt.setInt(2, id);
      pstmt.executeUpdate();
      pstmt.close();

      System.out.println("Teléfono actualizado para el contacto ID " + id + ".");
    } catch (SQLException a) {
      System.out.println("Error SQL: " + a.getMessage());
    } catch (Exception ex) {
      System.out.println("Error: " + ex.getMessage());
    }
  }

  public void eliminarContacto(int id) {
    try {
      verificarExisteContacto(id);

      PreparedStatement psEmails = conn.prepareStatement("DELETE FROM emails WHERE
      ↪  contacto_id = ?");
      psEmails.setInt(1, id);
      psEmails.executeUpdate();
      psEmails.close();

      PreparedStatement psDirs =
          conn.prepareStatement("DELETE FROM direcciones WHERE contacto_id = ?");
      psDirs.setInt(1, id);
      psDirs.executeUpdate();
      psDirs.close();

      PreparedStatement psContacto = conn.prepareStatement("DELETE FROM contactos
      ↪  WHERE id = ?");
      psContacto.setInt(1, id);
      psContacto.executeUpdate();
      psContacto.close();

```

```
217       System.out.println("Contacto ID " + id + " y sus datos relacionados fueron
          ↪  eliminados.");
218     } catch (SQLException a) {
219       System.out.println("Error SQL: " + a.getMessage());
220     } catch (Exception ex) {
221       System.out.println("Error: " + ex.getMessage());
222     }
223   }
224
225   public int crearEmail(int contactoId, String email) {
226     int idGenerado = -1;
227     try {
228       validarTexto(email, "email");
229       verificarExisteContacto(contactoId);
230
231       String sql = "INSERT INTO emails (contacto_id, email) VALUES (?, ?)";
232       PreparedStatement pstmt = conn.prepareStatement(sql,
          ↪  Statement.RETURN_GENERATED_KEYS);
233       pstmt.setInt(1, contactoId);
234       pstmt.setString(2, email.trim());
235       pstmt.executeUpdate();
236
237       ResultSet rs = pstmt.getGeneratedKeys();
238       if (rs.next()) {
239         idGenerado = rs.getInt(1);
240       }
241       rs.close();
242       pstmt.close();
243     } catch (SQLException a) {
244       System.out.println("Error SQL: " + a.getMessage());
245     } catch (Exception ex) {
246       System.out.println("Error: " + ex.getMessage());
247     }
248     return idGenerado;
249   }
250
251   public void leerEmails(int contactoId) {
252     try {
253       verificarExisteContacto(contactoId);
254
255       PreparedStatement pstmt =
256           conn.prepareStatement("SELECT id, email FROM emails WHERE contacto_id =
              ↪  ?");
257       pstmt.setInt(1, contactoId);
258       ResultSet rs = pstmt.executeQuery();
259
260       while (rs.next()) {
261         System.out.println("Email ID " + rs.getInt("id") + ": " +
            ↪  rs.getString("email"));
262       }
263       rs.close();
264       pstmt.close();
265     } catch (SQLException a) {
266       System.out.println("Error SQL: " + a.getMessage());
267     } catch (Exception ex) {
268       System.out.println("Error: " + ex.getMessage());
269     }
270   }
```

```
271
272   public void actualizarEmail(int id, String nuevoEmail) {
273     try {
274       validarTexto(nuevoEmail, "email");
275
276       PreparedStatement pstmt = conn.prepareStatement("UPDATE emails SET email = ?
          ↪  WHERE id = ?");
277       pstmt.setString(1, nuevoEmail.trim());
278       pstmt.setInt(2, id);
279       int filas = pstmt.executeUpdate();
280       pstmt.close();
281
282       if (filas == 0) {
283         throw new SQLException("No existe el email con id " + id + ".");
284       }
285       System.out.println("Email ID " + id + " actualizado.");
286     } catch (SQLException a) {
287       System.out.println("Error SQL: " + a.getMessage());
288     } catch (Exception ex) {
289       System.out.println("Error: " + ex.getMessage());
290     }
291   }
292
293   public void eliminarEmail(int id) {
294     try {
295       PreparedStatement pstmt = conn.prepareStatement("DELETE FROM emails WHERE id =
          ↪  ?");
296       pstmt.setInt(1, id);
297       int filas = pstmt.executeUpdate();
298       pstmt.close();
299
300       if (filas == 0) {
301         throw new SQLException("No existe el email con id " + id + ".");
302       }
303       System.out.println("Email ID " + id + " eliminado.");
304     } catch (SQLException a) {
305       System.out.println("Error SQL: " + a.getMessage());
306     } catch (Exception ex) {
307       System.out.println("Error: " + ex.getMessage());
308     }
309   }
310
311   public int crearDireccion(int contactoId, String direccion) {
312     int idGenerado = -1;
313     try {
314       validarTexto(direccion, "direccion");
315       verificarExisteContacto(contactoId);
316
317       String sql = "INSERT INTO direcciones (contacto_id, direccion) VALUES (?, ?)";
318       PreparedStatement pstmt = conn.prepareStatement(sql,
          ↪  Statement.RETURN_GENERATED_KEYS);
319       pstmt.setInt(1, contactoId);
320       pstmt.setString(2, direccion.trim());
321       pstmt.executeUpdate();
322
323       ResultSet rs = pstmt.getGeneratedKeys();
324       if (rs.next()) {
325         idGenerado = rs.getInt(1);
```

```
326         }
327         rs.close();
328         pstmt.close();
329       } catch (SQLException a) {
330         System.out.println("Error SQL: " + a.getMessage());
331       } catch (Exception ex) {
332         System.out.println("Error: " + ex.getMessage());
333       }
334       return idGenerado;
335     }
336
337     public void leerDirecciones(int contactoId) {
338       try {
339         verificarExisteContacto(contactoId);
340
341         PreparedStatement pstmt =
342             conn.prepareStatement("SELECT id, direccion FROM direcciones WHERE
                ↪  contacto_id = ?");
343         pstmt.setInt(1, contactoId);
344         ResultSet rs = pstmt.executeQuery();
345
346         while (rs.next()) {
347           System.out.println("Dirección ID " + rs.getInt("id") + ": " +
              ↪  rs.getString("direccion"));
348         }
349         rs.close();
350         pstmt.close();
351       } catch (SQLException a) {
352         System.out.println("Error SQL: " + a.getMessage());
353       } catch (Exception ex) {
354         System.out.println("Error: " + ex.getMessage());
355       }
356     }
357
358     public void actualizarDireccion(int id, String nuevaDireccion) {
359       try {
360         validarTexto(nuevaDireccion, "direccion");
361
362         PreparedStatement pstmt =
363             conn.prepareStatement("UPDATE direcciones SET direccion = ? WHERE id = ?");
364         pstmt.setString(1, nuevaDireccion.trim());
365         pstmt.setInt(2, id);
366         int filas = pstmt.executeUpdate();
367         pstmt.close();
368
369         if (filas == 0) {
370           throw new SQLException("No existe la dirección con id " + id + ".");
371         }
372         System.out.println("Dirección ID " + id + " actualizada.");
373       } catch (SQLException a) {
374         System.out.println("Error SQL: " + a.getMessage());
375       } catch (Exception ex) {
376         System.out.println("Error: " + ex.getMessage());
377       }
378     }
379
380     public void eliminarDireccion(int id) {
381       try {
```

```
382       PreparedStatement pstmt = conn.prepareStatement("DELETE FROM direcciones WHERE
          ↪  id = ?");
383       pstmt.setInt(1, id);
384       int filas = pstmt.executeUpdate();
385       pstmt.close();
386
387       if (filas == 0) {
388         throw new SQLException("No existe la dirección con id " + id + ".");
389       }
390       System.out.println("Dirección ID " + id + " eliminada.");
391     } catch (SQLException a) {
392       System.out.println("Error SQL: " + a.getMessage());
393     } catch (Exception ex) {
394       System.out.println("Error: " + ex.getMessage());
395     }
396   }
397 }
```

## 5.2. Clase Main

```
1  package agenda;
2
3  public class Main {
4
5    public static void main(String[] args) {
6      System.out.println("--- Conexión y creación de tablas ---");
7      GestorContactos gestor = new GestorContactos();
8      gestor.crearTablas();
9
10     System.out.println("\n--- Inserción de registros iniciales ---");
11     int c1 = gestor.crearContacto("Juan", "Pérez", "2614001122");
12     gestor.crearEmail(c1, "juan.perez@mail.com");
13     gestor.crearDireccion(c1, "San Martín 120, Mendoza");
14
15     int c2 = gestor.crearContacto("María", "Gómez", "2615112233");
16     gestor.crearEmail(c2, "maria.gomez@mail.com");
17     gestor.crearDireccion(c2, "Las Heras 450, Ciudad");
18
19     int c3 = gestor.crearContacto("Carlos", "López", "2616223344");
20     gestor.crearEmail(c3, "carlos.lopez@mail.com");
21     gestor.crearDireccion(c3, "Belgrano 780, Godoy Cruz");
22
23     int c4 = gestor.crearContacto("Ana", "Martínez", "2617334455");
24     gestor.crearEmail(c4, "ana.martinez@mail.com");
25     gestor.crearDireccion(c4, "Sarmiento 310, Guaymallén");
26
27     int c5 = gestor.crearContacto("Lucía", "Fernández", "2618445566");
28     gestor.crearEmail(c5, "lucia.fernandez@mail.com");
29     gestor.crearDireccion(c5, "Mitre 650, Luján de Cuyo");
30
31     System.out.println("5 contactos iniciales cargados.");
32
33     System.out.println("\n--- Listado completo ---");
34     gestor.leerContactos();
35
36     System.out.println("\n--- Modificación de datos de un contacto ---");
37     gestor.actualizarTelefono(c1, "2619998877");
```

```
38      int mailExtra = gestor.crearEmail(c1, "juan.trabajo@mail.com");
39      int dirExtra = gestor.crearDireccion(c1, "Colón 200, Ciudad");
40      gestor.actualizarEmail(mailExtra, "jperez@trabajo.com");
41      gestor.actualizarDireccion(dirExtra, "Colón 250, Ciudad");
42      gestor.leerContactos();
43
44      System.out.println("\n--- Eliminación de un contacto ---");
45      gestor.eliminarEmail(mailExtra);
46      gestor.eliminarDireccion(dirExtra);
47      gestor.eliminarContacto(c2);
48      gestor.leerContactos();
49
50      System.out.println("\n--- Manejo de excepciones ---");
51      new GestorContactos("jdbc:sqlite:/carpeta_inexistente/agenda.sqlite");
52      gestor.crearEmail(c1, "");
53      gestor.crearContacto("", "Apellido", "2610000000");
54      gestor.actualizarTelefono(999, "2610000000");
55      gestor.eliminarContacto(999);
56
57      gestor.cerrarConexion();
58    }
59  }
```

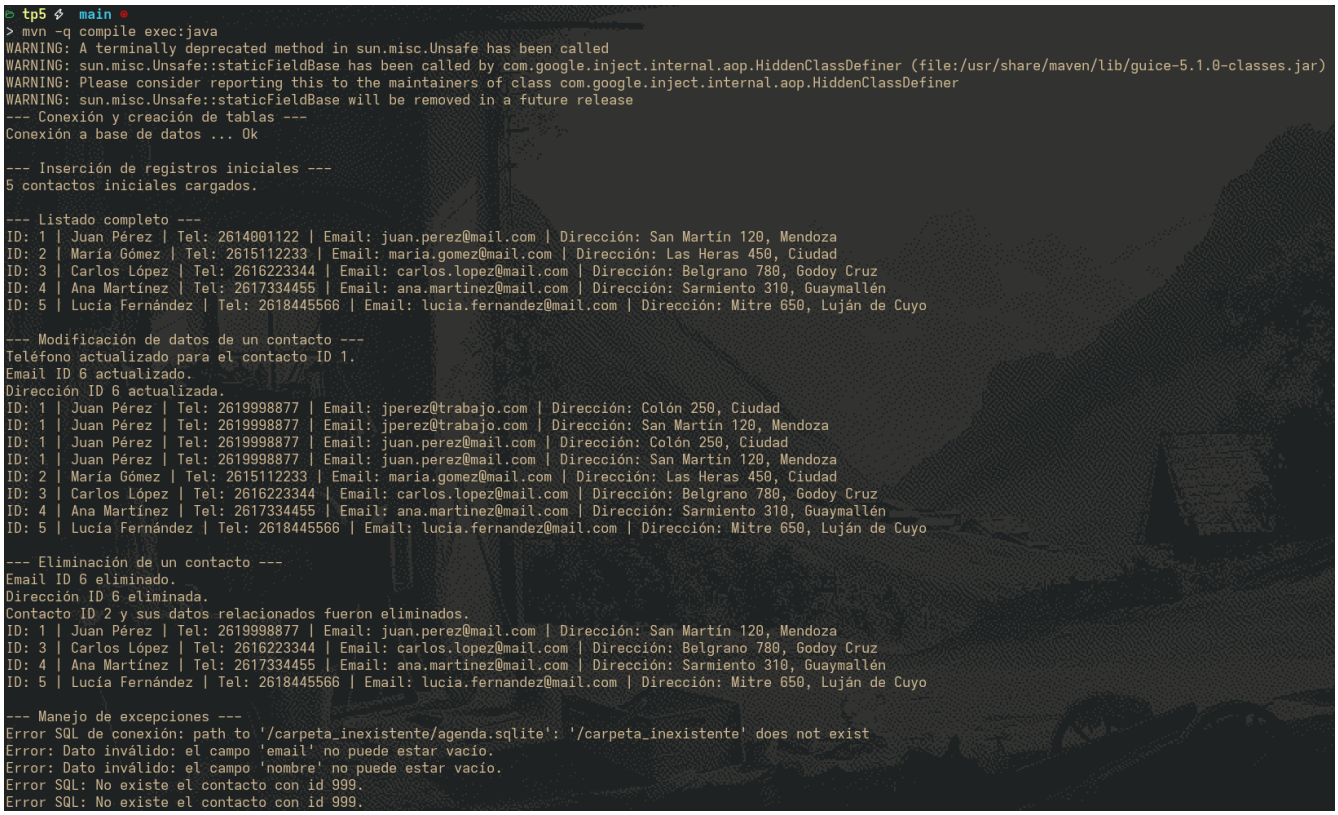



Figura 1: Salida de Main.java